

Sprint Documentation

Sprint # 1	Start Date: 30/09/25	Final Date: 06/10/25
Projetc Name:	civitas-backend	
Ano: 4º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Felipe Pacheco Bianchini		
Wendell Alves Nascimento		
Giovanni da Silva Nascimento		

Sprint Backlog

Task#	Description	Start Date	Final date
12	Criação do CRUD de Secretaria	30/09/25	06/10/25

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
12	Implementar os métodos padrão do CRUD para a entidade Secretaria, contemplando as seguintes operações: -Cadastrar -Alterar -Inativar e ativar registro -Listar	Felipe Pacheco Bianchini, Wendell Alves Nascimento, Giovanni da Silva Nascimento	In Progress	10	10

Sprint Description

Implementar os métodos padrão do CRUD para a entidade Secretaria, contemplando as seguintes operações:

-Cadastrar

-Alterar

-Inativar e ativar registro

-Listar

1. INTRODUÇÃO

Este relatório apresenta o desenvolvimento realizado na Sprint 01 – Task 12 do projeto Civitas, focado na implementação completa das funcionalidades CRUD (Create, Read, Update, Delete) para a entidade Secretaria. O desenvolvimento seguiu a arquitetura e os padrões estabelecidos no projeto, utilizando ASP.NET Core 9.0 e Entity Framework Core como principais tecnologias.

2. DESCRIÇÃO DO PRODUTO DESENVOLVIDO

2.1. Visão Geral

Foi implementado um conjunto completo de operações para gerenciamento de Secretarias municipais, permitindo cadastrar, consultar, alterar, excluir e alterar o status (ativo/inativo) dos registros. A implementação seguiu o modelo de camadas com separação de responsabilidades entre Controllers, Services, Repositories e Models.

2.2. Funcionalidades Implementadas

1. **Cadastro de Secretarias** - Permite a inserção de novos registros com todos os campos necessários;
2. **Alteração de Dados** - Possibilita a atualização de informações em registros existentes;
3. **Consulta Completa** - Lista todos os registros de secretarias disponíveis;
4. **Consulta por ID** - Recupera detalhes de uma secretaria específica;
5. **Exclusão de Registros** - Remove permanentemente uma secretaria do banco de dados;
6. **Alteração de Situação** - Alterna o status entre ativo e inativo sem necessidade de atualização completa.

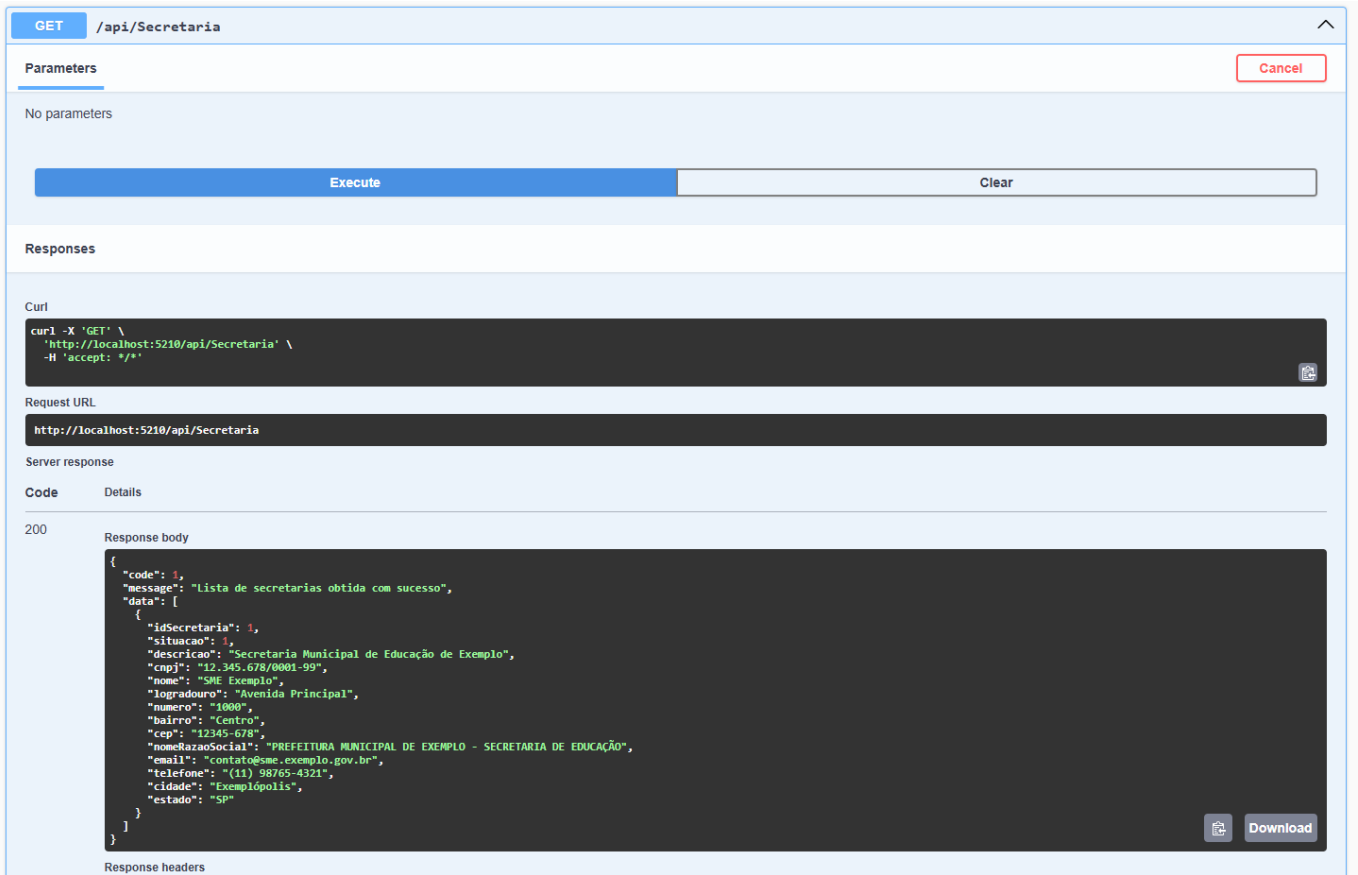
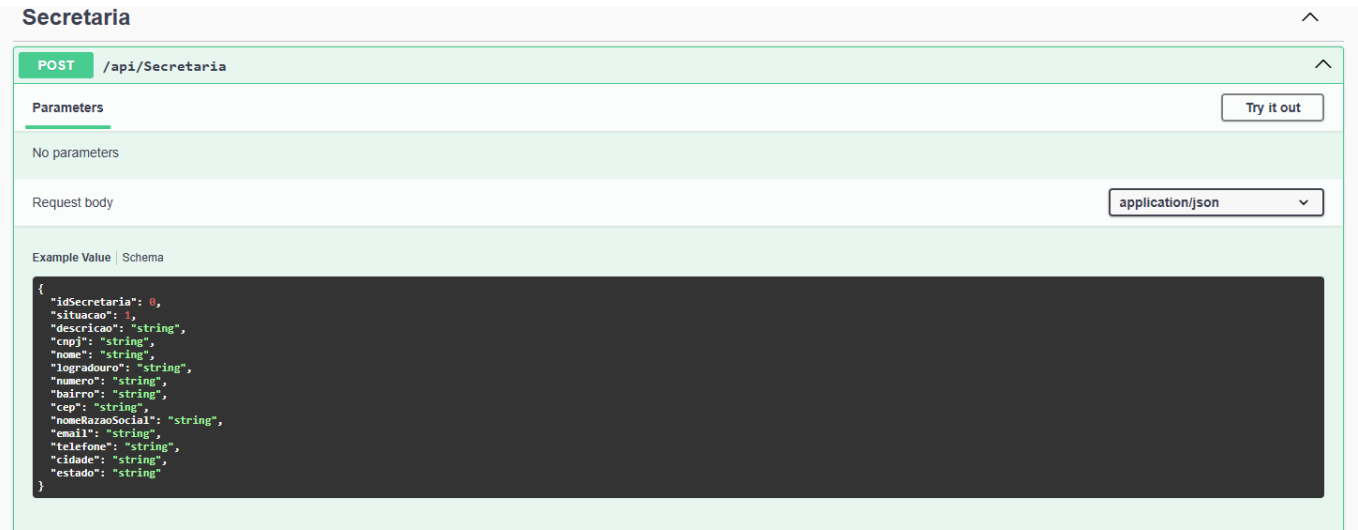
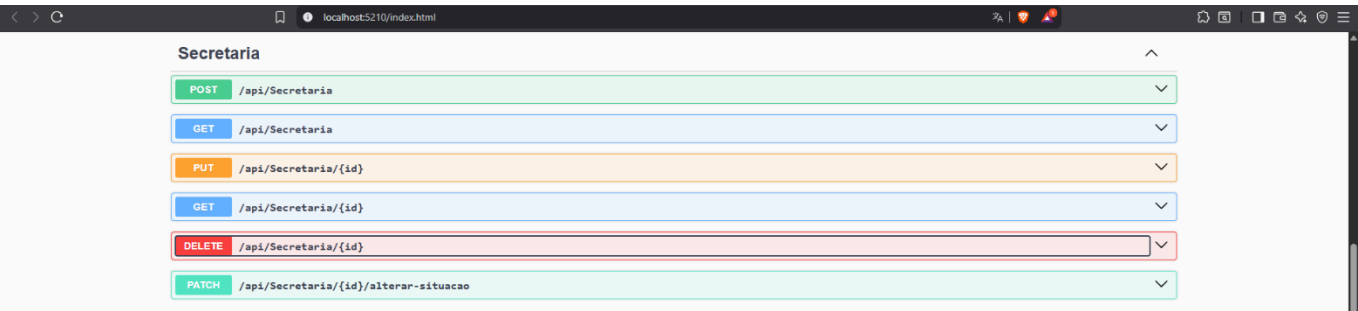
2.3. Estrutura de Dados

A entidade Secretaria foi implementada com os seguintes atributos:

- IdSecretaria (int): Chave primária
- Situacao (int): Status do registro (Ativo/Inativo)
- Descricao (string): Descrição da secretaria
- Cnpj (string): Documento identificador
- Nome (string): Nome abreviado da secretaria
- Logradouro (string): Endereço - rua/avenida
- Numero (string): Número do endereço
- Bairro (string): Bairro da localização
- Cep (string): Código postal
- NomeRazaoSocial (string): Nome oficial completo
- Email (string): Contato eletrônico
- Telefone (string): Contato telefônico

- Cidade (string): Município de localização
- Estado (string): Unidade federativa

3. CAPTURAS DE TELA



4. ASPECTOS POSITIVOS E APRENDIZADOS

4.1. Aspectos Técnicos Bem-Sucedidos

1. **Aplicação Consistente da Arquitetura:** A implementação seguiu rigorosamente o padrão arquitetural estabelecido no projeto, com adequada separação de responsabilidades entre camadas.
2. **Tratamento Adequado de Validações:** Foram implementadas validações em múltiplos níveis, garantindo integridade dos dados desde a entrada na API até a persistência no banco de dados.
3. **Migrations Efetivas:** A criação e aplicação de migrations ocorreu sem problemas, com adequada manutenção da estrutura do banco de dados, incluindo ajustes necessários para campos formatados.
4. **Injeção de Dependências:** Todas as dependências foram corretamente registradas no container IoC, garantindo baixo acoplamento e alta testabilidade.

4.2. Aprendizados

1. **Tratamento de Campos Formatados:** Foi necessário ajustar o tamanho dos campos no banco de dados para acomodar dados com formatação (como CNPJ com pontos e traços), resultando em melhor usabilidade.
2. **Importância do AutoMapper:** A implementação destacou a necessidade de configurar corretamente o AutoMapper para conversão entre entidades e DTOs, evitando erros em tempo de execução.
3. **Padronização de Respostas:** A utilização consistente da classe Response para padronização das respostas da API melhorou significativamente a experiência de consumo dos endpoints.
4. **Validações Progressivas:** A implementação de validações em diferentes camadas (Controller, Service, Repository) permitiu um tratamento mais granular e preciso de erros.

5. LIMITAÇÕES E JUSTIFICATIVAS

Não houve funcionalidades planejadas que não puderam ser implementadas nesta sprint. Todos os requisitos foram atendidos conforme especificação.

No entanto, identificou-se a necessidade de ajustes não previstos inicialmente:

1. **Ajuste de Tamanho de Campos:** Foi necessário aumentar o tamanho dos campos CNPJ (de 14 para 18 caracteres), CEP (de 8 para 10 caracteres) e Telefone (de 15 para 20 caracteres) para acomodar dados com formatação. Esta necessidade só foi identificada durante os testes de integração.
2. **Configuração Adicional do AutoMapper:** A configuração específica para mapeamento entre Secretaria e SecretariaDTO não estava prevista inicialmente, sendo adicionada após identificação de erro durante os testes.

6. TESTES REALIZADOS

6.1. Testes de Integração

Foram realizados testes de integração via Swagger e curl para validar o funcionamento dos endpoints:

1. **Teste de Criação (POST):** Verificação da criação de novos registros com dados válidos e tratamento adequado de dados inválidos.
2. **Teste de Consulta (GET):** Validação da recuperação de registros individuais e listas completas.

3. **Teste de Atualização (PUT):** Confirmação da capacidade de alterar todos os campos de registros existentes.
4. **Teste de Alteração de Status (PATCH):** Verificação da funcionalidade de ativação/inativação de registros.
5. **Teste de Exclusão (DELETE):** Validação da remoção de registros e tratamento adequado para tentativas de exclusão de registros inexistentes.

6.2. Exemplo de Teste via curl

Exemplo de requisição POST para criação de secretaria:

```
curl -X 'POST' \  
  
'http://localhost:5210/api/Secretaria' \  
  
-H 'accept: */*' \  
  
-H 'Content-Type: application/json' \  
  
-d '{  
  
  "idSecretaria": 0,  
  
  "situacao": 1,  
  
  "descricao": "Secretaria Municipal de Educação de Exemplo",  
  
  "cnpj": "12.345.678/0001-99",  
  
  "nome": "SME Exemplo",  
  
  "logradouro": "Avenida Principal",  
  
  "numero": "1000",  
  
  "bairro": "Centro",  
  
  "cep": "12345-678",  
  
  "nomeRazaoSocial": "PREFEITURA MUNICIPAL DE EXEMPLO - SECRETARIA DE EDUCAÇÃO",  
  
  "email": "contato@sme.exemplo.gov.br",  
  
  "telefone": "(11) 98765-4321",  
  
  "cidade": "Exemplópolis",  
  
  "estado": "SP"  
  
}'
```

7. CONCLUSÃO

A Sprint 01 – Task 12 foi concluída com sucesso, entregando todas as funcionalidades CRUD previstas para a entidade Secretaria. O código desenvolvido segue os padrões estabelecidos no projeto, com adequada separação de responsabilidades, validações robustas e tratamento de exceções.

As pequenas adaptações necessárias durante o desenvolvimento (ajuste de tamanho de campos e configuração do AutoMapper) foram prontamente identificadas e resolvidas, garantindo a qualidade final do produto.

A API REST implementada atende completamente aos requisitos definidos, fornecendo endpoints intuitivos e respostas padronizadas que facilitarão o consumo por parte do frontend e de outras aplicações.

Assinaturas